

Programmation Orientée Objet

Jusqu'à présent nous avons utilisé des **objets** dont le type est prédéfini par Python : `int`, `float`, `bool`, `str`, `list`, `tuple`, `dict`...

Nous avons même appris à utiliser des **méthodes** associées à ces types, comme `ma_liste.append(valeur)`.

Paradigme impératif

Une façon de concevoir, modéliser et programmer une solution à un problème s'appelle un **paradigme de programmation**.

Celui que nous utilisons depuis la première est le **paradigme impératif**. C'est à dire l'écriture de **séquences d'instructions** et de fonctions appelées ensuite à de multiples endroits du programme. Ce mode de programmation est utilisé pour des **programmes simples** car il devient vite lourd à gérer, écrire et déboguer.

Dans ce chapitre, nous allons aborder le **paradigme objet**. Il va nous permettre de créer de nouveaux types de données, appelées **classes**, possédant des **attributs et des méthodes**.

Exercice

Avec vos connaissances actuelles, comment représenteriez-vous une carte de jeu, le 7 de pique par exemple ?

La POO : Penser objet !

Une autre façon de **penser** notre programme est de considérer que nous allons manipuler **des objets** qui possèdent **certaines caractéristiques** et sur lesquels nous pouvons **agir**.

Python, comme d'autres langages (Java, C++, ...), est un langage **orienté objet**. La programmation orientée objet définit un ensemble d'objets qui **interagissent entre eux**. En d'autres termes : tout est objet.

Une variable de type `int` est en fait un **objet de type `int`** donc construit à partir de la **classe `int`**.

Chaque **type** correspond à une **classe**.

Avec la programmation orientée objet, on peut créer ses propres objets qui auront leurs propres caractéristiques et fonctions.

Elle n'améliore pas subitement la qualité de l'application comme par magie. En revanche, elle permet de :

- De faciliter l'implémentation de données complexes ou particulière
- Mieux organiser le code et augmenter la lisibilité du programme
- De faciliter les éventuelles modifications du programme
- D'avoir un programme plus modulable et réutilisable

C'est pour cela que les développeurs professionnels l'utilisent dans la plupart de leurs projets.

La classe est la définition d'un type d'objets.

L'encapsulation est le principe de cacher les détails d'implémentation de l'extérieur par une interface.

Notion d'objet et de classe

Dans le monde réel, on peut illustrer un objet par une voiture, un téléphone, un livre, etc. En Informatique, un objet n'est ni une simple variable ni une simple fonction mais un mélange des deux.

Dans notre exemple, une carte est vue comme un **objet** qui possède une valeur et une couleur : c'est ce que nous appellerons les **attributs** de l'objet.

On veut également pouvoir l'afficher, la comparer, accéder à ses données ou encore la modifier : c'est ce que nous appellerons les **méthodes** de l'objet.

Les attributs et les méthodes sont définies **dans la classe**.

Nous devons faire la différence entre le modèle de carte défini ci-dessus par ses attributs et méthodes, c'est ce que nous appellerons une **classe**, et une carte en particulier que nous appellerons **objet**.

Définition : objet

Un objet est défini par son **état** (ses paramètres, caractéristiques) dans des variables appelées **attributs**.

On implémente son **comportement** (ses actions) à l'aide de fonctions appelées **méthodes**.

Objet = un état + un comportement

Définition : classe

Le **type d'un ensemble d'objets** qui ont la même structure mais qui n'ont pas les mêmes valeurs est appelé **Classe**.

C'est dans cette classe que sont déclarés **les attributs et méthodes communs** à tous les objets.

On dit qu'un objet est une **instance de classe**

Classe = attributs + méthodes

On peut considérer par exemple la classe suivante :

- Bicyclette :
 - Ses caractéristiques sont : marque, type, nombre de vitesses, vitesse courante, couleur, etc.
 - son comportement est de : rouler, tourner, accélérer, changer de vitesse, freiner, ...

Les objets peuvent être `maBicyclette`, `taBicyclette` ou encore `bicyclette_bob` avec des valeurs d'états différentes.



Exercice

1. Lister les attributs et méthodes possibles pour une classe Rectangle.
2. Puis donner 2 exemples différents d'objets de type Rectangle

Attributs	Méthodes

Cycle de vie d'un objet

Chaque objet (instance) a un cycle de vie :

- **Construction** : création, initialisation des attributs et stockage en mémoire.
- **Utilisation** : changements d'état ou de comportement par exécution de méthodes

Pour créer un objet, il faut d'abord créer une classe !

Comme pour une maison, il faut avoir un plan avant de la construire. La classe décrit la **structure des objets**.

Après avoir réfléchi aux attributs et méthodes de(s) classe(s) nécessaires il est important de les représenter sous forme de **schéma** (diagramme de classes) afin d'avoir une vision claire de la structure de nos données

Carte
- valeur : int - couleur : str
+afficher() +valeur() +couleur() +modifier_valeur() +modifier_couleur()

La syntaxe de Python

En python, la définition d'une classe commence par le mot clé `class` suivi du **nom de la classe** (qui par convention commence par une majuscule) et enfin du caractère `:`

```
1 class Carte :  
2     """contenu de la classe pour implémenter des cartes"""
```

On peut maintenant alimenter le contenu de notre classe, ses méthodes (on veillera à ce que son contenu soit bien indenté):

Les méthodes de la classe sont en quelque sorte des **fonctions définies pour la classe et qui s'appliqueront aux objets**. Par conséquent la définition d'une méthode est similaire à celle d'une fonction : mot clé `def`.

Dans la classe, pour **nommer l'objet** auquel la méthode s'applique on utilisera le mot clé `self`, que l'on **précise dans les paramètres**.

Il existe des méthodes spéciales en Python qui ont leur nom encadrés par des double underscore `__`, comme le **constructeur** (méthode qui permet de créer des objets) nommé `__init__`.

Le constructeur

Dans tout langage de programmation, et donc en Python, on doit disposer d'une méthode particulière permettant de créer un objet à partir de sa classe. Par exemple pour les listes : `liste(4,1,3)`.

Donnons un exemple précis, si nous indiquons qu'une carte a une valeur et une couleur (nous définissons une structure, c'est à dire une classe). Il faut alors pouvoir dire qu'on veut créer une carte en particulier à partir de ce modèle, par exemple la carte 3 de carreau.

Un **constructeur** est une méthode particulière dont le rôle est de créer un nouvel objet.

Les constructeurs ont principalement deux tâches :

- réserver un **emplacement en mémoire** pour le nouvel objet,
- **initialiser les attributs** de cet objet, c'est-à-dire leur donner une valeur initiale

En Python le constructeur se nomme toujours `__init__`.

Exemple :

```

1 class Carte :
2
3     # les paramètres sont les données nécessaires
4     # pour créer une carte
5     def __init__(self, val, coul) :
6         '''
7         Entrées : self : l'objet Carte qui sera créé
8                   val (int) valeur de la carte
9                   coul (str) couleur de la carte
10        Sortie : None. initialise une carte
11        '''
12        # tests facultatifs
13        assert(isinstance(val, int)), 'val doit être un entier'
14        assert(isinstance(coul, str)), 'coul doit être un str'
15
16        # définition et initialisation des attributs
17        #self correspond à l'objet que l'on va créer
18        self.valeur = val
19        self.couleur = coul
20    # fin du constructeur

```

Le nombre de paramètres du constructeur peuvent varier selon les données nécessaires pour créer un objet.

Exercice

Quels sont les noms des attributs de cette classe ?

Créer un objet

Maintenant que nous avons un constructeur, nous pouvons **instancier** cette classe dans un programme pour créer un objet informatique représentant une carte en particulier et le stocker dans une variable.

```

1 # Carte(...) va appeler le constructeur __init__ de la classe Carte
2 # et va initialiser la variable carte1 qui deviendra un objet de type Carte
3 carte1 = Carte(3, "carreau")

```

Il ne faut pas oublier de passer autant de valeurs en paramètre qu'il y a dans la définition du constructeur (sans le `self` qui est la carte créée).

Exercice

Donner le code permettant d'instancier la dame de coeur.

Maintenant que l'on peut créer des instances de notre classe, on va lui ajouter des méthodes qui vont correspondre aux différentes actions que l'on peut effectuer sur les objets de type Carte.

Accesseurs et mutateurs

Il est possible d'accéder à la valeur des attributs d'un objet et même les modifier dans le programme principal. On dit que les attributs sont **publics**. Cela peut poser plusieurs problèmes :

- Possibilité d'erreurs non gérées comme l'affectation d'une valeur d'un mauvais type
- Si on souhaite modifier la structure de la classe, comme le nom d'un attribut par exemple.

Il est possible de rendre des attributs ou des méthodes **privés**.

Un attribut privé **ne pourra pas être appelé ou modifié en dehors de la classe**, il faudra appeler des méthodes de la classe pour le faire.

Nous admettrons que les attributs sont privés par défaut.

On va alors créer des méthodes de classe avec un but précis :

- des **accesseurs** (getters en anglais), permettant **d'accéder à la valeur de chaque attribut**
- des **mutateurs** (setters en anglais), permettant de **modifier les attributs**.

Ainsi, on va déconnecter le plus possible la structure (donc la classe) du reste du code. Le programme principal appellera les méthodes nécessaires ainsi les différentes erreurs seront donc gérées par la classe elle-même.

Et si l'on décide de modifier la structure, il y aura seulement à re-coder la(les) méthode(s) impactées. Cela facilite la maintenance et les améliorations.

Exercice

Compléter le code (ligne 2 et 5) du getter et setter pour l'attribut valeur de la classe (ces méthodes sont ajoutées à la classe, à la suite du constructeur)

```
1 def get_valeur(self) :  
2     return  
3  
4 def set_valeur(self, new_val) :  
5
```

Appeler une méthode sur un objet

Une méthode s'appelle différemment d'une fonction :

- l'objet sur laquelle elle s'applique est d'abord précisé (`carte1` par exemple)
- un `.` suit le nom de l'objet
- ensuite seulement vient la méthode et ses éventuels paramètres (sauf le `self` qui est justement l'objet précisé avant le point, la variable).

Nous avons déjà utilisé à de multiples reprises cette syntaxe en Python pour les listes par exemple.

Nous allons maintenant implémenter cette classe en entier puis créer une autre classe **Paquet** représentant un paquet de carte (elle utilisera donc la classe `Carte`).

L'objectif sera de créer un jeu de bataille fonctionnel grâce à la POO.